

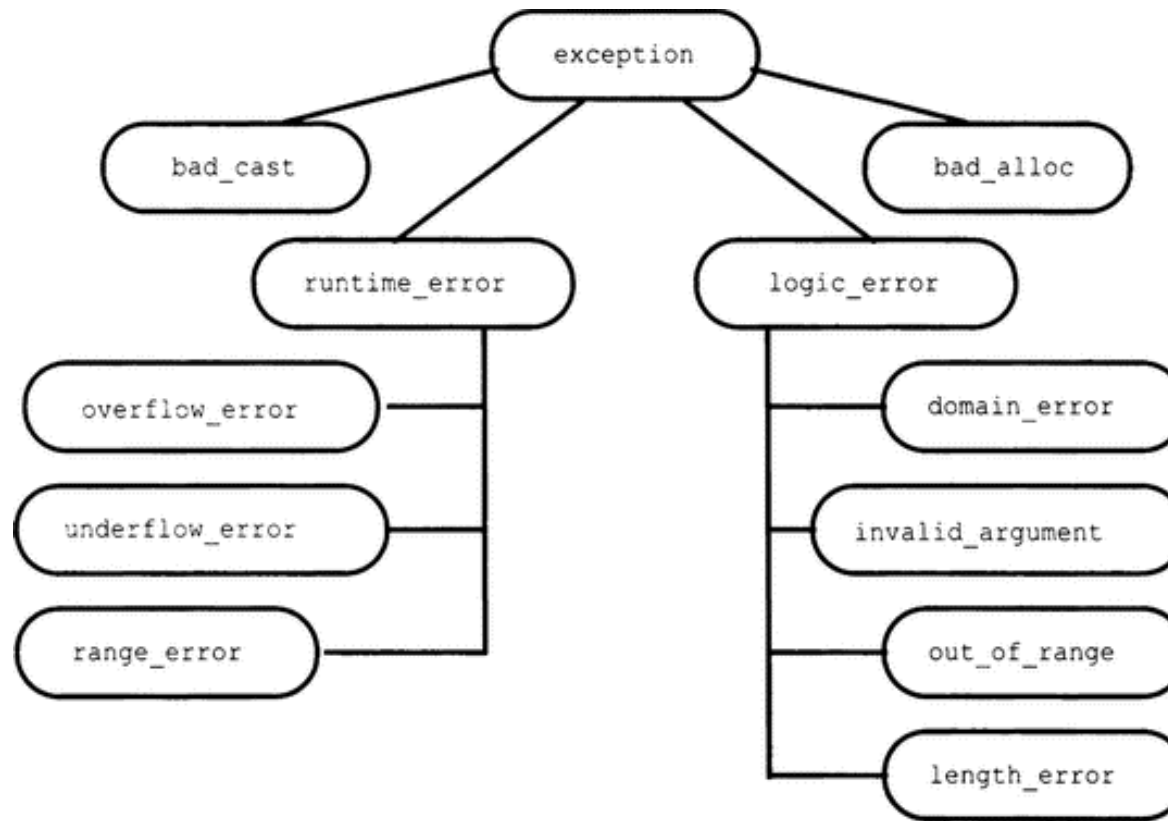
Семинар по C++ №1

Исключения

База

- Механизм обработки исключительных ситуаций
- Stack unwinding
- Не путать с RE
- Нужно следить за копированиями
- Два исключения одновременно приводят к RE

Иерархия исключений



Разрешается каст вверх в блоке catch

noexcept

- [Noexcept specifier](#) – часть сигнатуры функции. Если исключение летит из функции, помеченной noexcept, вызывается terminate.
- [Noexcept operator](#) – оператор, проверяющий, является ли выражение noexcept.
- [] обычно не помечены noexcept, хотя не могут кидать исключение, но могут вести к SegFault.

std::terminate

- std::terminate вызывается при необработанном исключении (и ещё в 11 случаях)
- set_terminate / get_terminate – установить/получить обработчик
- Сигнатура обработчика:
- `void ( *terminate_handler )()`
- std::current_exception(), std::rethrow_exception

std::uncaught_exception(s)

- Можно узнать, есть ли непоиманное исключение (std::uncaught_exception) или количество непоиманных исключений (std::uncaught_exceptions)
- Исключений может быть одновременно несколько, если деструктор вызывает функцию, которая кидает исключение, но деструктор сам его обрабатывает.

Гарантии безопасности

1. Гарантия отсутствия исключения

2. Строгая гарантия безопасности: исключение может возникнуть, но в этом случае всё будет возвращено к тому, как было до вызова соответствующей функции

3. Слабая гарантия безопасности: при возникновении ошибки мы не можем вернуть всё как было, но всё равно останемся в корректном состоянии

4. Отсутствие безопасности

Задача

- Реализуйте функцию `TransformIf(begin, end, p, f)`, которая принимает 2 указателя, задающие последовательность, унарный предикат `p` и функцию `f`, и применяет данную функцию ко всем элементам последовательности, для которых выполнен заданный предикат. Считается, что `f` модифицирует переданный элемент (принимает по ссылке), поэтому данная функция в отличие от `std::transform` работает in-place.
- Обозначим за `T` тип элементов последовательности. Для упрощения в этой задаче не используется move-семантика. Вы можете считать, что у `T` определен конструктор копирования и оператор присваивания.
- Ваша функция должна вести себя атомарно — либо полностью модифицировать последовательность, либо оставлять ее в исходном состоянии (при возникновении исключений). Для этого держите лог всех измененных элементов (для которых выполнен предикат) и при исключении восстанавливайте последовательность в исходное состояние при помощи лога. Не копируйте всю последовательность заранее.
- Если точнее, функция должна гарантировать следующее:
- Если `p` и/или `f` бросает исключение, то функция должна оставить последовательность в первоначальном виде (и бросить то же исключение).
- Если при копировании `T` возникает исключение, а `p` и `f` исключений не бросают, то функция должна отработать полностью (и не бросать исключений).
- Если исключения возникают и при копировании `T`, и в `p` и/или `f`, функция оставляет последовательность в неопределенном состоянии и бросает одно из исключений.

Function try block

- Можно оборачивать целую функцию в try block:

```
int main() try {  
    throw 1;  
} catch (int x) {  
    std::cout << "Caught int " << x << '\n';  
}
```

Статический объект

- Если деструктор бросается в конструкторе статического объекта внутри функции, то конструктор будет вызываться каждый раз при вызове функции до тех пор, пока конструктор не завершится без исключения.
- Посмотреть [тут](#).

Альтернативы исключениям

1. Возвращать код ошибки
2. Передавать ссылку на объект ошибки в функцию (как в [std::filesystem](#))
3. Выставлять errno
4. std::optional / std::variant
5. Писать на Rust-е
6. C++23 expected

Что почитать

- <https://tproger.ru/articles/isklyucheniya-v-s-garantii-bezopasnosti-i-spezifikacii/#part1>
- [Исключения изнутри](#)
- [Как исключения хранятся в памяти](#)